Halo EVM 엔진 통합 분석

개요

Halo는 `octane/evmengine` 패키지를 통해 Geth(Go-Ethereum) 엔진과 직접 통합됩니다. 이 문서는 EVM 엔진의 구조, Geth 통합 방식, 그리고 페이로드 처리 과정을 상세히 분석합니다.

EVM 엔진 패키지 구조

1. 디렉토리 구조

```
octane/evmengine/
├── keeper/ # EVM 엔진 키퍼 (상태 관리)
├── module/ # Cosmos SDK 모듈 통합
└── types/ # 타입 정의 및 변환 함수
```

2. 핵심 컴포넌트

2.1 EVM 엔진 키퍼

```go
type Keeper struct {
cdc codec.BinaryCodec
storeService store.KVStoreService
headTable ExecutionHeadTable // 실행 헤드 저장
```

```
withdrawalTable WithdrawalTable // 출금 관리
engineCl ethclient.EngineClient // Geth 엔진 클라이언트
txConfig client.TxConfig
voteProvider types.VoteExtensionProvider
eventProcs []types.EvmEventProcessor // EVM 이벤트 프로세서들
cmtAPI comet.API
addrProvider types.AddressProvider
feeRecProvider types.FeeRecipientProvider
buildDelay time.Duration
buildOptimistic  bool
maxWithdrawalsPerBlock  uint64

// 최적화된 페이로드 관리
mutablePayload  struct {
sync.Mutex
ID engine.PayloadID
Height  uint64
UpdatedAt time.Time
}
}
```

2.2 인터페이스 정의

```
// 주소 제공자 인터페이스
```

```
type AddressProvider interface {
LocalAddress() common.Address // 로컬 검증자의 이더리움 주소
}


// 수수료 수신자 제공자 인터페이스
type FeeRecipientProvider interface {
LocalFeeRecipient() common.Address // 로컬 수수료 수신 주소
VerifyFeeRecipient(proposedFeeRecipient common.Address) error  // 수수료 수신자 검
}
```

Geth 통합 방식

1. Engine API 클라이언트

EVM 엔진은 Geth의 Engine API를 통해 통신합니다:

```
type EngineClient interface {
// 포크초이스 업데이트 (새 헤드 설정 및 페이로드 빌드 시작)
ForkchoiceUpdatedV3(ctx context.Context, fcs engine.ForkchoiceStateV1, attrs *en
// 페이로드 가져오기
GetPayloadV3(ctx context.Context, payloadID engine.PayloadID) (*engine.Execution
// 새 페이로드 푸시 (검증 및 실행)
NewPayloadV3(ctx context.Context, payload engine.ExecutableData, blobHashes []co
// 이벤트 로그 필터링
```

```
FilterLogs(ctx context.Context, q ethereum.FilterQuery) ([]ethtypes.Log, error)
}
```

2. 페이로드 빌드 과정

2.1 포크초이스 상태 설정

```
func (k *Keeper) startBuild(ctx context.Context, appHash common.Hash, timestamp
// 현재 실행 헤드 가져오기
head, err := k.getExecutionHead(ctx)
if  err != nil {
return engine.ForkChoiceResponse{}, errors.Wrap(err, "latest execution block")
}

// CometBFT는 즉시 최종성을 가지므로 head/safe/finalized는 모두 동일
fcs := engine.ForkchoiceStateV1{
HeadBlockHash: headHash,
SafeBlockHash: headHash,
FinalizedBlockHash: headHash,
}

// 페이로드 속성 설정
attrs := &engine.PayloadAttributes{
```

```
Timestamp: ts,
Random: headHash, // 헤드 블록 해시를 randao로 사용
SuggestedFeeRecipient: k.feeRecProvider.LocalFeeRecipient(), // 수수료 수신자
Withdrawals: withdrawals, // 출금 목록
BeaconRoot: &appHash, // 앱 해시를 비콘 루트로 사용
}

// Geth에 포크초이스 업데이트 요청
resp, err := k.engineCl.ForkchoiceUpdatedV3(ctx, fcs, attrs)
return resp, err
}
```

2.2 페이로드 가져오기

```
// PrepareProposal에서 페이로드 가져오기
var payloadResp *engine.ExecutionPayloadEnvelope
err = retryForever(ctx, func(ctx context.Context) (bool, error) {
var err error
payloadResp, err = k.engineCl.GetPayloadV3(ctx, payloadID)
if isUnknownPayload(err) {
return false, err // 중단, 재시도 안함
} else if err != nil {
log.Warn(ctx, "Preparing proposal failed: get evm payload (will retry)", err)
```

```go
return false, nil  // 재시도
}
return true, nil  // 완료
})
```

3. 페이로드 검증 및 실행

3.1 페이로드 푸시

```go
func  pushPayload(ctx context.Context, engineCl ethclient.EngineClient, payload
sdkCtx := sdk.UnwrapSDKContext(ctx)
appHash, err := cast.EthHash(sdkCtx.BlockHeader().AppHash)
if  err != nil {
return engine.PayloadStatusV1{}, err
}

// Geth에 페이로드 푸시 (새 헤드로 마킹)
status, err := engineCl.NewPayloadV3(ctx, payload, blobHashes, &appHash)
if  err != nil {
return engine.PayloadStatusV1{}, errors.Wrap(err, "new payload")
}

return  status, nil
```

```
}
```

3.2 페이로드 상태 처리

```
func isSyncing(status engine.PayloadStatusV1) bool {
return status.Status == engine.SYNCING || status.Status == engine.ACCEPTED
}

func isInvalid(status engine.PayloadStatusV1) (bool, error) {
if status.Status != engine.INVALID {
return false, nil
}

valErr := "nil"
if status.ValidationError != nil {
valErr = *status.ValidationError
}

hash := "nil"
if status.LatestValidHash != nil {
hash = status.LatestValidHash.Hex()
}
```

```
return true, errors.New("payload invalid", "validation_err", valErr, "last_valid
}
```

페이로드 처리 과정

1. 데이터 타입 변환

1.1 Protobuf ↔ Geth 변환

```
// Geth ExecutableData를 Protobuf로 변환
func PayloadToProto(p *engine.ExecutableData) (*ExecutionPayloadDeneb, error) {
if p == nil {
return nil, errors.New("nil payload")
}

var baseFeePerGas [32]byte
p.BaseFeePerGas.FillBytes(baseFeePerGas[:])

var withdrawals []Withdrawal
for _, w := range p.Withdrawals {
withdrawal, err := WithdrawalToProto(w)
if err != nil {
return nil, errors.Wrap(err, "withdrawal to proto")
}
```

```
withdrawals = append(withdrawals, withdrawal)
}

return &ExecutionPayloadDeneb{
ParentHash: Hash(p.ParentHash),
FeeRecipient: Address(p.FeeRecipient),
StateRoot: Hash(p.StateRoot),
ReceiptsRoot: Hash(p.ReceiptsRoot),
LogsBloom: p.LogsBloom,
PrevRandao: Hash(p.Random),
BlockNumber: p.Number,
GasLimit: p.GasLimit,
GasUsed: p.GasUsed,
Timestamp: p.Timestamp,
ExtraData: p.ExtraData,
BaseFeePerGas: baseFeePerGas[:],
BlockHash: Hash(p.BlockHash),
Transactions: p.Transactions,
Withdrawals: withdrawals,
BlobGasUsed: *p.BlobGasUsed,
ExcessBlobGas: *p.ExcessBlobGas,
}, nil
}
```

```go
// Protobuf를 Geth ExecutableData로 변환
func PayloadFromProto(p *ExecutionPayloadDeneb) (engine.ExecutableData, error)
withdrawals := make([]*ethtypes.Withdrawal, 0) // Geth는 빈 배열 필요 (null 아님)
for _, w := range p.Withdrawals {
withdrawals = append(withdrawals, WithdrawalFromProto(w))
}

transactions := p.Transactions
if transactions == nil {
transactions = make([][]byte, 0) // Geth는 빈 배열 필요 (null 아님)
}

return engine.ExecutableData{
ParentHash: common.Hash(p.ParentHash),
FeeRecipient: common.Address(p.FeeRecipient),
StateRoot: common.Hash(p.StateRoot),
ReceiptsRoot: common.Hash(p.ReceiptsRoot),
LogsBloom: p.LogsBloom,
Random: common.Hash(p.PrevRandao),
Number: p.BlockNumber,
GasLimit: p.GasLimit,
GasUsed: p.GasUsed,
```

```
Timestamp: p.Timestamp,
ExtraData: p.ExtraData,
BaseFeePerGas: new(big.Int).SetBytes(p.BaseFeePerGas),
BlockHash: common.Hash(p.BlockHash),
Transactions: transactions,
Withdrawals: withdrawals,
BlobGasUsed: &p.BlobGasUsed,
ExcessBlobGas: &p.ExcessBlobGas,
}, nil
}
```

1.2 출금(Withdrawal) 처리

```
func WithdrawalToProto(w *ethtypes.Withdrawal) (Withdrawal, error) {
if w == nil {
return Withdrawal{}, errors.New("nil withdrawal")
} else if w.Validator != 0 {
return Withdrawal{}, errors.New("non-zero validator index") // 검증자 인덱스는 0이
}

return Withdrawal{
Index: w.Index,
Address: Address(w.Address),
```

```
AmountGwei: w.Amount,
}, nil
}

func  WithdrawalFromProto(w Withdrawal) *ethtypes.Withdrawal {
return &ethtypes.Withdrawal{
Index: w.Index,
Address: common.Address(w.Address),
Amount: w.AmountGwei,
}
}
```

2. 페이로드 검증

2.1 제안된 페이로드 검증

```
func (k *Keeper) parseAndVerifyProposedPayload(ctx context.Context, msg *types.M
// 권한 확인
if  msg.Authority != authtypes.NewModuleAddress(types.ModuleName).String() {
return engine.ExecutableData{}, errors.New("invalid authority")
}

// 페이로드 파싱
payload, err := types.PayloadFromProto(msg.ExecutionPayloadDeneb)
```

```
if  err != nil {
return engine.ExecutableData{}, errors.Wrap(err, "unmarshal proto payload")
}

// 출금 검증
eligibleWithdrawals, err := k.eligibleWithdrawals(ctx, height)
if  err != nil {
return engine.ExecutableData{}, errors.Wrap(err, "eligible withdrawals")
}

if !withdrawalsEqual(payload.Withdrawals, eligibleWithdrawals) {
return engine.ExecutableData{}, errors.New("mismatch with eligible withdrawals")
}

// 실행 증인 확인 (허용되지 않음)
if  payload.ExecutionWitness != nil {
return engine.ExecutableData{}, errors.New("witness not allowed in payload")
}

// 수수료 수신자 검증
if  err := k.feeRecProvider.VerifyFeeRecipient(payload.FeeRecipient);  err != ni
return engine.ExecutableData{}, errors.Wrap(err, "verify proposed fee recipient"
}
```

```
// 부모 해시 및 블록 높이 검증
head, err := k.getExecutionHead(ctx)
if  err != nil {
return engine.ExecutableData{}, errors.Wrap(err, "latest execution block")
}

if  payload.Number != head.GetBlockHeight()+1 {
return engine.ExecutableData{}, errors.New("invalid proposed payload number")
}

headHash, err := head.Hash()
if  err != nil {
return engine.ExecutableData{}, err
}

if  payload.ParentHash != headHash {
return engine.ExecutableData{}, errors.New("invalid proposed payload parent hash
}

// 타임스탬프 검증
minTimestamp := head.GetBlockTime() + 1
maxTimestamp, err := umath.ToUint64(sdk.UnwrapSDKContext(ctx).BlockTime().Unix()
```

```go
if err != nil {
return engine.ExecutableData{}, err
}

if payload.Timestamp < minTimestamp || payload.Timestamp > maxTimestamp {
return engine.ExecutableData{}, errors.New("invalid payload timestamp")
}

// Randao 다이제스트 검증 (현재는 부모 해시 사용)
if payload.Random != headHash {
return engine.ExecutableData{}, errors.New("invalid payload random")
}

return payload, nil
}
```

3. EVM 이벤트 처리

3.1 이벤트 수집

```go
func (k *Keeper) evmEvents(ctx context.Context, blockHash common.Hash) ([]types.
return fetchProcEvents(ctx, k.engineCl, blockHash, sortIndex, k.eventProcs...)
}
```

```go
func fetchProcEvents(ctx context.Context, cl ethclient.EngineClient, blockHash

var all []ethtypes.Log

for _, proc := range procs {

// EVM 이벤트 가져오기 (네트워크 에러 시 재시도)

err := retryForever(ctx, func(ctx context.Context) (bool, error) {

addresses, topics := proc.FilterParams()

logs, err := cl.FilterLogs(ctx, ethereum.FilterQuery{

BlockHash: &blockHash,

Addresses: addresses,

Topics: topics,

})

if err != nil {

log.Warn(ctx, "Failed fetching evm events (will retry)", err, "proc", proc.Name(

return false, nil // 재시도

}



all = append(all, logs...)

return true, nil // 완료

})

if err != nil {

return nil, err

}

}
```

```go
// 로그를 인덱스 순으로 정렬 (결정론적 순서)
sortFunc(all)

// 로그 검증 및 변환
var resp []types.EVMEvent
for _, l := range all {
topics := make([][]byte, 0, len(l.Topics))
for _, t := range l.Topics {
topics = append(topics, t.Bytes())
}

event := types.EVMEvent{
Address: l.Address.Bytes(),
Topics: topics,
Data: l.Data,
}

if err := event.Verify(); err != nil {
return nil, errors.Wrap(err, "verify evm events")
}

resp = append(resp, event)
```

```go
}

return resp, nil
}
```

3.2 이벤트 전달

```go
func (s msgServer) deliverEvents(ctx context.Context, height uint64, blockHash
// 주소별 프로세서 매핑
procs := make(map[common.Address]types.EvmEventProcessor)
for _, proc := range s.eventProcs {
addrs, _ := proc.FilterParams()
for _, addr := range addrs {
procs[addr] = proc
}
}

for _, event := range events {
elog, err := event.ToEthLog()
if err != nil {
return err
}
```

```
proc, ok := procs[elog.Address]
if !ok {
return  errors.New("unknown log address [BUG]")
}

// 스토어 분기 (처리 실패 시 롤백)
sdkCtx := sdk.UnwrapSDKContext(ctx)
branchMS := sdkCtx.MultiStore().CacheMultiStore()
branchCtx := sdkCtx.WithMultiStore(branchMS)

// 패닉을 에러로 변환하여 이벤트 전달
if  err := catch(func() error {
return  proc.Deliver(branchCtx, blockHash, event)
});  err != nil {
log.Warn(ctx, "Delivering EVM log event failed", err, "name", proc.Name(), "heig
continue  // 에러 시 상태 쓰기 안함
}

branchMS.Write() // 성공 시 상태 쓰기
}

return  nil
```

```
}
```

최적화 기능

1. 최적화된 페이로드 빌드

```
func (k *Keeper) PostFinalize(ctx sdk.Context) error {
if !k.buildOptimistic {
return nil // 최적화 빌드 비활성화
}

// 다음 제안자인지 확인
isNext, err := k.isNextProposer(ctx, height)
if err != nil {
log.Warn(ctx, "Next proposer failed, skipping optimistic EVM payload build", err
return nil
} else if !isNext {
return nil // 다음 제안자가 아니면 건너뛰기
}

// 최적화된 EVM 페이로드 빌드 시작
fcr, err := k.startBuild(ctx, appHash, timestamp, nextHeight)
if err != nil {
```

```
log.Warn(ctx, "Starting optimistic build failed", err)
return nil
}

// 페이로드 ID 저장
k.setOptimisticPayload(*fcr.PayloadID, nextHeight)
return nil
}
```

2. 에러 복구 메커니즘

```
// 네트워크 에러에 대한 무한 재시도
func retryForever(ctx context.Context, fn func(ctx context.Context) (bool, err
backoff := expbackoff.New(ctx)
for {
innerCtx, cancel := context.WithTimeout(ctx, retryTimeout)
ok, err := fn(innerCtx)
cancel()
if ctx.Err() != nil {
return errors.Wrap(ctx.Err(), "retry canceled")
} else if err != nil {
return err
} else if !ok {
```

```
backoff() // 지수 백오프로 재시도
continue
}

return nil
}
}
```

3. Blob 처리

```
// Blob 커밋먼트에서 해시 생성
func blobHashes(commitments [][]byte) ([]common.Hash, error) {
if len(commitments) > maxBlobsPerBlock {
return nil, errors.New("too many blobs", "max", maxBlobsPerBlock, "actual", len
}

hasher := sha256.New()
resp := make([]common.Hash, 0, len(commitments))
for _, commitment := range commitments {
commitment48, err := cast.Array48(commitment)
if err != nil {
return nil, err
}
```

```
resp = append(resp, kzg4844.CalcBlobHashV1(hasher, (*kzg4844.Commitment)(&commit
}

return  resp, nil
}
```

Cosmos SDK 모듈 통합

1. 모듈 정의

```
type AppModule struct {
AppModuleBasic
keeper *keeper.Keeper
}

func (m AppModule) InitGenesis(ctx sdk.Context, cdc codec.JSONCodec, raw json.Ra
var  data types.GenesisState
cdc.MustUnmarshalJSON(raw, &data)

// 제네시스 실행 헤드 삽입
if  err := m.keeper.InsertGenesisHead(ctx, data.ExecutionBlockHash);  err != nil
panic(errors.Wrap(err, "insert genesis head"))
```

```
}
}
```

2. 의존성 주입

```
func ProvideModule(in ModuleInputs) (ModuleOutputs, error) {
// 이벤트 프로세서 언래핑
var eventProcs []types.EvmEventProcessor
for _, proc := range in.EventProcs {
eventProcs = append(eventProcs, proc)
}

k, err := keeper.NewKeeper(
in.Cdc,
in.StoreService,
in.EngineCl,
in.TXConfig,
in.AddrProvider,
in.FeeRecProvider,
in.Config.GetMaxWithdrawalsPerBlock(),
eventProcs...,
)
```

```
if  err != nil {
return ModuleOutputs{}, err
}

m := NewAppModule(in.Cdc, k)

return ModuleOutputs{
EngEVMKeeper: k,
Module: m,
Hooks: staking.StakingHooksWrapper{StakingHooks: keeper.Hooks{}},
}, nil
}
```

요약

Halo의 EVM 엔진 통합은 다음과 같은 특징을 가집니다:

1. **직접 통합**: Geth Engine API를 통한 직접적인 EVM 통합
2. **페이로드 관리**: Protobuf와 Geth 형식 간 완전한 변환 지원
3. **이벤트 처리**: EVM 이벤트의 결정론적 수집 및 전달
4. **최적화**: 다음 제안자 예측을 통한 미리 빌드
5. **복구 메커니즘**: 네트워크 에러에 대한 강력한 재시도 로직
6. **검증**: 다중 레벨 페이로드 검증으로 보안성 확보
7. **모듈화**: Cosmos SDK와의 깔끔한 통합

이러한 설계를 통해 Halo는 CometBFT 합의와 EVM 실행을 효과적으로 결합하여 이더리움 호환성과 크로스체인 기능을 동시에 제공합니다.